I don't understand Python's Asyncio (pocoo.org)

376 points by ingve on Oct 30, 2016 | hide | past | favorite | 207 comments

jerf on Oct 30, 2016 | next [–]

I think I've made the point a few times that I don't like this style of programming at all, because the coroutine layer turns into an Inner Platform [1] replicating all the control-flow structures the original language has, which then has to integrate with the original language which causes more than twice the complexity to emerge. But it's hard to bash together an example of how that happens in a comment, when it's all faked up and easy to dismiss. This is a great example of the complexity that can emerge. Some of that is incidental and will be fixed in future releases, but some of that is pretty fundamental, such as the initial list of all the sorts of things you need to learn about to use asyncio. Languages like Erlang or Go that have the event loop simply embedded into the language from day one have a much shorter list of such things you have to know, and it does all the things that this complicated list of asyncio concepts do.

But I will also admit some of this complexity is definitely Python-specific. I've been using Python since 1.5.2 was still the version you'd be most likely to encounter and I've liked it as a language for a while, but one release at a time, the language has been getting more and more complicated. By the time asyncio came around, the language was quite complicated, and sticking asyncio on top while integrating it with everything else is really a mess. Python has too darned many features at this point. I don't know exactly when it jumped the shark feature-wise, because individually they all make sense, but the sum total has gotten quite unwieldy. Watching new programmers try to learn Python, a language I used to suggest to such people as a good language to start with, has been a bit dispiriting lately.

[1]: https://en.wikipedia.org/wiki/Inner-platform_effect

RodericDay on Oct 30, 2016 | parent | next [–]

I don't think the number of features is too problematic if it is properly gated. Python having 'http.server' or a Fractions module or tkinter or a matrix multiplication operator comes off as a pleasant surprise, it doesn't hamper the onboarding experience.

However, async, typing syntax, multiple string formatting syntaxes, and so on all do. The ship is being helmed irresponsibly in this regard. Letting decorator and keyword asyncio coexist, as well as threads and multiprocess, just feels like a huge mess.

pmontra on Oct 30, 2016 | root | parent | next [–]

I agree that the features are not a problem. A large standard library and lots of packages are actually very welcome. The problem is the very design of the language, from the beginning.

I inherited a couple of Python 2.7 projects recently and after many years of toying with it I'm finally using Python seriously. Well, the syntax of the language is a mix of strange decisions. Part functional, part object oriented, not well designed in any of those parts. The so old school looking special __functions__. The annoying syntax errors one gets when refactoring and moving code around courtesy of the useless : and significant spacing. The even more annoying indentation errors when copy pasting code from the editor to the interactive interpreter. ["a", "b"].join(".") and ".".split("a.b") or other permutations, I still can't remember which one is correct. It's not a very inviting home to live in, a kind of non euclidean space.

It's strange that Python got where it is today. Maybe it's because among the languages that were big 10 years ago it's more immediate to work with than Java an C++, because PHP is despised by many professionals especially pre v7 when it was really ugly, because nobody bothered to build and maintain a numpy for Ruby and because JavaScript become big only recently. I feel that Python is good enough to do almost everything but not particularly nice to look at.

However I want to end on a lighter note quoting Matz. "No language can be perfect for everyone. I tried to make Ruby perfect for me, but maybe it's not perfect for you. The perfect language for Guido van Rossum is probably Python."

goatlover on Oct 31, 2016 | root | parent | next [–]

I feel that Python is more readable than most languages, and nice to look at it. I also like the magic functions. And whitespace never bothered me a second. I have the same amount of whitespace errors as I do forgetting a semicolon in PHP, which is infrequently. I don't know why you would consider JavaScript a better option with all its warts.

But you are right about the Python ecosystem. It is massive in the scientific and statistical computing. The other scripting languages outside of R aren't in the same ballpark in that domain.

pmontra on Oct 31, 2016 | root | parent | next [–]

> why you would consider JavaScript a better option with all its warts

I don't. Until recently there were so many ways to define a class that I was constantly forgetting how I did it last time. This is an order of magnitude worse than the split join thing. And all the

wierd things we had to do to work around the callback hell. And the verbosity of function everywhere.

At least it is moving into the right direction with the last iterations of the language, which are mitigating those problems.

Perseids on Oct 31, 2016 | root | parent | prev | next [–]

> It's not a very inviting home to live in, a kind of non euclidean space.

The surface of the earth is my favorite place to live on, but maybe you prefer infinite planes?

(Earth's surface is non euclidean, parallel lines intersect. And, yeah, I know, we don't live *in* the plane. For a more accurate example, this whole universe is non-euclidean, see space distortions around black holes.)

upquark on Oct 31, 2016 | root | parent | prev | next [–]

> not a very inviting home to live in, a kind of non euclidean space

Hmm, i've always felt the exact opposite, that Python feels just right syntax-wise (this after years of C/C++, Perl, bash, JavaScript, etc).

What would be your example of a language residing in "euclidean space"?

pmontra on Oct 31, 2016 | root | parent | next [–]

Ruby is not perfect but feels more logical and consistent.

"a.b".split(".") and ["a", "b"].join(".")

Ruby is bad if you want to do some functional programming but I find this logical: there are functional languages for that, which in turn are bad at object orientation. That's fine.

What I don't understand is Python doing OO by making us declare self in the method definitions as if it were a functional language that must explicitly carry around the state. Every other OO language knows how to handle self (JS is following a different OO model.) Python object orientation looks very low level. I was passing around self in C (no ++) to simulate OO: the pointer to the struct with the object data, function pointers and parent classes. Let's say that Python is very close to its implementation in C, but why?

repsilat on Oct 31, 2016 | root | parent | next [–]

> but why?

In Python (almost?) any unqualified identifier you see in an expression is either a builtin function or it's defined/imported somewhere else in the file. I find Ruby a little stressful by comparison (without even getting into the awful cultural approval of defining the names of things procedurally, ensuring you'll *never* find where they came from...)

The lack of parens on function calls also adds uncertainty for me. I know in Python you can overload `__getattr__` and introduce just as much magic, but for the most part I can be confident that `a.b` doesn't do anything too crazy. That's the general trend for me -- Python is almost relentlessly boring, with a few little surprises that stick out mostly because everything else is so plain and sensible. Ruby is just a little crazier everywhere, partly because the language is a bit more eccentric and partly because the people who use it are all Ruby programmers :-)

pmontra on Oct 31, 2016 | root | parent | next [–]

My why was about why one should design a language in that way and not think harder and make it look better, but in the early days of Python there weren't many other OO languages around, so I can understand that it could have been natural to go somewhat low level and mimic C (even with all those __s). Maybe some language went more high level and died because of that. Python passed the test of time so that (self): probably was a good idea at the end of the 80s / early 90s.

In the case of Ruby, you can't name a function (which is a method) without executing it. That's why () don't matter much. The optional () also make Ruby a good language to write DSLs. By the way, if you want to get a reference to a method, you must prepend it with a &, pretty much like in C. Ha! :-) This demonstrates that every language has its quirks. Or you can call a method by sending a message to its object like object.send(:method) using a symbol named after the method. That's more or less a reference to it, which can be metaprogrammed because symbols can be built from strings ("something".to_sym). Is that the "defining the names of things procedurally" you don't like? On the other side, I find stressful that in Python you have to enumerate all your imports, like in Java. It's the same in Ruby, but I'm almost always programming in Rails and it auto imports everything. All those imports in Django and Web2py are tiresome. I got naming clashes with Rails only a couple of times in 10 years but I missed imports many times in Django yesterday.

I learned Python after Ruby. My trajectory was BASIC in the 80s, Pascal, C, Perl, a little TCL, Java and JavaScript since their beginning, Ruby since Rails, a little Python and PHP a few years later, much less Perl now and almost nothing of what preceded it, some Elixir. I keep using Ruby and JS, I'm using Python now. I insist that compared to Pascal, Java and Ruby Python looks illogical and unnecessarily complex, but I can understand why people with a different history can feel like Ruby is eccentric. I remember when I demoed it to some PHP developer many years ago, he said it was like writing in English, which was surprising because it is not how it looks to me, but it felt flattering.

cma on Oct 31, 2016 | root | parent | next [–]

I like the __'s, when listing methods you can immediately scan it and ignore all the built-in stuff if you want, to see what is special about this object.

In order to get "split" in ruby for a sequence, at least a one time, hopefully cleaned up by now, you end up mixing in some huge number of methods and made any method list in the console impossible to read.

upquark on Oct 31, 2016 | root | parent | prev | next [–]

> "a.b".split(".") and ["a", "b"].join(".")

Seems like a minor point to me, really. Sure, join could've been a member function of the list class, but that would prevent applying it to arbitrary iterables, no? In other words, delimiter.join(items) is more general than items.join(delimiter), because in the latter join must be a member function of the class of items or its ancestor class, and you won't be able to apply it to other iterable objects.

I haven't had much interaction with Ruby, but from my limited experience the syntax felt strictly less intuitive than Python. The only other languages where syntax felt more intuitive to me than Python is the ML family (with derivatives like OCaml, etc).

klibertp on Oct 31, 2016 | root | parent | next [–]

> Seems like a minor point to me, really.

All the points raised here are minor and, mostly, subjective. Inability to comprehend this is the only reason such discussions are being repeated over and over again. It's utterly useless to discuss what syntax feels "natural" to whom - it's entirely dependent on what other languages (types of syntax, really) you already know.

There are some characteristics of syntax that we can discuss, for example, how large it is or what characters it tends to mainly use, but discussing these is apparently less fun than saying that something "feels illogical to me".

upquark on Oct 31, 2016 | root | parent | next [–]

I meant "minor point" relative to the other points. It's all relative and subjective, of course, I never said otherwise.

Also, doesn't seem like you read the *actual* argument following the first sentence :)

klibertp on Nov 1, 2016 | root | parent | next [–]

> Also, doesn't seem like you read the actual argument following the first sentence :)

Yeah, I was mainly referring to the @pmontra comments, like this one: "I find stressful that in Python you have to enumerate all your imports" and similar.

> Sure, join could've been a member function of the list class, but that would prevent applying it to arbitrary iterables, no?

Not true if your language supports multimethods, see Dylan, Common Lisp and CLOS or Nim and Julia for examples. Also not true if you add the "join" method high enough in the class hierarchy: as an example, in Pharo Smalltalk you have the following hierarchy: ProtoObject -> Object -> Collection -> SequencableCollection -> ArrayedCollection -> String with the "reduce" method being declared on Collection class (reduce being the easiest way to implement "join").

So in short: no. There are many interesting languages which implement various interesting techniques which solve various problems (like the so-called "Expression problem"); it's good to know about them even if you're not going to use them all that much (or at all).

> delimiter.join(items) is more general than items.join(delimiter)

But then you loose the ability to join a collection with a separator not being a String or you need to implement join even higher in the hierarchy (on Object most probably).

> The only other languages where syntax felt more intuitive to me

Yeah, this is what I'm campaigning against. This notion of "intuitiveness" is completely useless and is dependent on how your intuition was formed. All syntaxes of programming languages are artificial and man-made - there is nothing "natural" about them at all. In other words, they are all similarly alien and only get "intuitive" with practice. Programmers usually learn only a single syntax flavor during their careers, which is why they don't realize that the "intuitiveness" is just a function of familiarity. Learning some of the other kinds of syntax is good because it lets you observe how your "intuition" is shifting and changing in the process.

upquark on Nov 1, 2016 | root | parent | next [–]

Your annoyingly patronizing tone aside, I'll try to address what you're saying.

> ...the class hierarchy: as an example, in Pharo Smalltalk you have the following hierarchy: ProtoObject -> Object -> Collection -> SequencableCollection -> ArrayedCollection -> String

Seems ridiculously over-engineered to me, but whatever, let's keep going..

> with the "reduce" method being declared on Collection class (reduce being the easiest way to implement "join").

'reduce' and 'join' are very different things. one is a generic function (aka fold, also exists in python as 'reduce'), the other is a string concatenation method that takes an iterable and produces a string. the latter can be implemented via the former, but they're not the same thing. no one's stopping you from using 'reduce' in Python instead of the built-in string member function 'join', btw.

> So in short: no. There are many interesting languages which implement various interesting techniques which solve various problems

Ugh, i give up :)

pekk on Oct 31, 2016 | root | parent | prev | next [–]

Whatever you prefer the syntax for join and split to be, I prefer the opposite, but even if you quote Matz that doesn't mean your way is the right one.

lixquid on Oct 31, 2016 | root | parent | next [–]

I don't think they were quoting Matz as justification for why Python is wrong, they were quoting Matz to justify why what they don't like about Python isn't the wrong way.

The whole point is that every language has something that someone doesn't like about it and every language is "good enough" for some subset of people. Instead of attacking what points everyone does or does not like about X, it's instead better to learn from each other and take the best parts.

pmontra on Oct 31, 2016 | root | parent | next [–]

Exactly. This is what I believe Matz was thinking.

wslh on Oct 30, 2016 | root | parent | prev | next [–]

> Letting decorator and keyword asyncio coexist, as well as threads and multiprocess, just feels like a huge mess.

That's the main issue. You can work with asyncio and threads at the same time but you need to know very well how the frameworks work.

dom0 on Oct 30, 2016 | root | parent | next [–]

Raw multi-threading was always a "blowing your feet of" feature in almost any way imaginable. That's true for Python as well and not only for use together with asyncio (but in this context even more so, I

agree).

No one should write production MT code based on pthread-esque abstractions in any language without knowing very very well what they're doing.

Fiahil on Oct 31, 2016 | root | parent | next [–]

I think you're right. I have python threads and processes meshed together in production code, and it's not pretty to look at. Sometimes it would crash for no reason, like this morning, when a job got stuck because one of its child process refused to die properly, thus blocking the join.

lmm on Oct 31, 2016 | root | parent | prev | next [–]

Due to the GIL and resulting coarse-grained nature of Python threading, it was very easy to write Python code that accidentally worked most of the time.

dom0 on Oct 31, 2016 | root | parent | next [–]

Interpreter instructions in CPython are atomic (due to the GIL, actually) and calling into extension functions is a single instruction (incidentally). This makes it impossible to corrupt built-in data structures, even if no locks are used. In some simple cases this is sufficient, esp. if no one's going to run it with !CPython...

This is also neat for other reasons (think sandboxing).

rocqua on Oct 30, 2016 | root | parent | prev | next [–]

In this case, it seems like things were pulled into the standard library to quickly. This then lead to conflicting ideas in the standard library.

rumcajz on Oct 30, 2016 | parent | prev | next [–]

Why can't Python have something like http://libmill.org ?

If it's doable in C, why not in Python?

rdtsc on Oct 30, 2016 | root | parent | next [–]

It has those. It as the greenlet library and used by eventlet and gevent. Many use those, and they've been there for many years. But Guido and others decided that is not acceptable. So we've gone the Twisted route (because everyone know Twisted is easy and fun) and now we have yields, co-routines, futures, awaits and all the other mess.

WaxProlix on Oct 31, 2016 | root | parent | next [–]

Lots of conversation between Armin and a few others about these approaches in reddit's /r/python thread for this article. I happen to think that gevent got a *ton* right, and the consensus seems to be that this 'new' approach is sort of half-baked, at least vis-a-vis python implementation.

https://www.reddit.com/r/Python/comments/5a6gmv/i_dont_under...

sadgit on Oct 31, 2016 | root | parent | prev | next [–]

Totally agree. Its a damn shame, too, it fit so elegantly into a programming style that looked very analgous to the threading library. Look at Tornado's StackContext to see the kind of mess that callpack passing creates. Its a gc nightmare.

afrancis on Oct 31, 2016 | root | parent | prev | next [–]

Although I am not aware of any recent development, you should look at Stackless Python which was inspired by Limbo. Stackless Python has lightweight threads (stacklets) and synchronous channels a la various Bell labs family of languages (include Go). The greenlets that Gevent uses comes from Stackless.

webmaven on Oct 31, 2016 | root | parent | next [–]

How does Stackless CPython[1] compare with Stackless PyPy[2]?

[1] http://www.stackless.com/

[2] http://doc.pypy.org/en/latest/stackless.html

afrancis on Oct 31, 2016 | root | parent | next [–]

Although I have used both CPython and Stackless.py, the specific differences are fading from my memory. While working on a Go style select() for stackless.py I recall there were minor differences in how the runnable list is represented and accessed. However this is really an implementation detail. For the most part, they are pretty much the same.

The biggest difference I remember in the current stackless.py implementation, the move to "continuelets" resulted in the inability to pickle "complex" stackless tasklets. So it becomes more

difficult to stop a tasklet in one thread and restart it in another. Maybe the ability to control the recursion depth (including getting rid of it) may also be gone in stackless.py.

mzs on Oct 30, 2016 | root | parent | prev | next [–]

http://libmill.org/documentation.html

```
  Libmill runs in following environments:

  Microarchitecture: x86_64, ARM
  Compiler: gcc, clang
  Operating system: Linux, OSX, FreeBSD, OpenBSD, NetBSD, DragonFlyBSD

  Whether it works in different environments is not known - please, do report any successes or
failures to the project mailing list.
```

danappelxx on Oct 31, 2016 | root | parent | next [–]

He was not suggesting to adopt libmill, only to adopt a similar concurrency scheme.

mzs on Nov 2, 2016 | root | parent | next [–]

I wasn't saying that either, I was saying libmill did not do it in (pure) C:

https://github.com/sustrik/libmill/blob/master/libmill.h

It only works with gcc and clang now, no longer with windows, and is sprinkled with asm()s. That doesn't fly with the CPython is portable. I don't know how these go-like features could be created with ANSI C, but I'd love to be proven wrong.

gigatexal on Oct 30, 2016 | root | parent | prev | next [–]

Whoa thanks for this, definitely going to check it out.

plq on Oct 30, 2016 | prev | next [–]

The general sentiment in this thread seems to be that Guido et al "blessed" the Twisted way, it's a pity and now using asyncio and friends is the only way to suspend a Python stack in a concurrent program besides using OS-level threads.

This is not correct. Green threads, as a programming paradigm, are just a sub-optimal (but cheaper) way of doing preemptive multitasking. Yes, switch is explicit, but user code can't know what will switch. So you are not supposed to treat green threads any differently from OS threads. ie you still need your mutexes, semaphores etc. if you want to avoid race conditions. Again, that's because the caller doesn't know whether a function will yield execution to the next queued task at some point. Guido explains this point pretty well in one of his PyCon keynotes.

So green threads may be great but they don't bring anything new to the table.

However, Twisted-style concurrency (aka cooperative multitasking) is a different paradigm. In Twisted, you know that you only have one thread running at a time, so you actually don't need any thread synchronization primitives when accessing non-local state. This simplifies things a great deal. Yes, not having to spawn a thread for every single concurrent IO operation has other great benefits, but that's not the reason why CPython now has a blessed event loop -- it's cooperative-multitasking-the-paradigm.

Before asyncio, there was no standard way of doing cooperative multitasking. Now there is and it's baked right into the language. Use it if you like it. If not, the old ways of doing things work just fine in Python 3.

I'll admit that the concurrency model in Python 3.4 was not perfect. However, what we have with Python 3.5 and up looks quite polished.

int_19h on Oct 30, 2016 | parent | next [–]

Another issue with green threads is that they usually do not work very well (or at all) across API, much less ABI, boundaries.

Basically, the moment your control flow is calling into some shared library, you probably want C API at that boundary for the sake of portability and stability. Exposing what is an, essentially, promise-based system that way is not hard. Better yet, if the other side has some analogous construct, you can map to that. But how do you do it with green threads?

Even platforms that have an OS-wide unified green thread primitive, like Win32 fibers, which would presumably solve this problem, find their use very lackluster, because many languages and frameworks just plain don't support fibers correctly. Even CLR tried to do it once and dropped the feature; forget about Python etc.

ntoshev on Oct 31, 2016 | parent | prev | next [–]

I disagree that the code can't know what will switch with green threads. They switch on I/O and when explicitly requested. An example of using this to get rid of concurrency control:
http://www.underengineering.com/2014/05/22/DIY-NoSql/

If you want to make sure that there are no context switching in certain part of your code, you can do it ASSERT-style, something like:

```
# enable this only if you use atomic, so a new module that should be imported before gevent
in_transaction = False
if __debug__:
    import greenlet
    old_switch=greenlet.greenlet.switch
    class _greenlet(greenlet.greenlet):
        def switch(*args, **kwargs):
            if in_transaction:
                raise Exception('Switching context during / atomic')
            old_switch(*args, **kwargs)
    setattr(greenlet, 'greenlet', _greenlet)

@contextmanager
def atomic():
    """
        Ensure that a function or a block of code is atomic, raise exception if it's not

        Usage:

        @atomic()
        def myTransaction(...):
            ...

        or

        with atomic():
            ...
    """
    global in_transaction
    in_transaction = True
    yield
    in_transaction = False
```

plq on Oct 31, 2016 | root | parent | next [–]

And how is this not a mutex?

ntoshev on Oct 31, 2016 | root | parent | next [–]

This only runs in the debug version of the code. I hope your mutexes run in production, too.

lmm on Oct 31, 2016 | root | parent | next [–]

So you rely on hoping your tests exercise all possible paths, because if not you get silent race conditions in prod?

ntoshev on Oct 31, 2016 | root | parent | next [–]

If you want you can run it in production, too (this would give you a warning, won't prevent race conditions, but there is almost no performance penalty). The point was this is very different from a mutex.

danappelxx on Oct 31, 2016 | parent | prev | next [–]

Have you seen libmill[0]? Single-threaded coroutines - no mutexes, semaphores, etc. We're using it in Zewo[1] with great success.

[0]: http://libmill.org [1]: https://github.com/Zewo/Zewo

fnl on Oct 30, 2016 | parent | prev | next [–]

Maybe the real issue with Twisted/asyncio is that it requires that *all* your code is "asyncio-ready". A bit like lock-free programming and only using linked lists.

That said, it's great that Python has something like this in its stdlib now, a bit like Node, Akka, and Go. But maybe asyncio needs a great big warning sign: "Use this only if you know why you need it and understand the impact it will have on your application's architecture."

rlpb on Oct 30, 2016 | root | parent | next [–]

> Maybe the real issue with Twisted/asyncio is that it requires that all your code is "asyncio-ready".

I don't think this is true. It's easy enough to ask for something that isn't "asyncio-ready" to run in a real thread. Give it a function call. It will run it in a thread, and give you a Future for when it's done. See https://docs.python.org/3/library/asyncio-eventloop.html#exe... for details.

Sure, it's a pain because you don't get the benefit that asyncio gives you for that part of code, but isn't at all an "all or nothing" proposition.

rdtsc on Oct 31, 2016 | parent | prev | next [–]

> ie you still need your mutexes, semaphores etc. if you want to avoid race conditions [...] In Twisted, you know that you only have one thread running at a time, so you actually don't need any thread synchronization primitives when accessing non-local state.

I disagree (unless you have a toy example or demo). As the software grows it becomes peppered with yields at the top level. Everything start yield -- authentication is an yield, launching a background job is an yield, writing to the databases. At that point you might find that some shared data has to be protected as well from concurrent IO requests so you still need semaphores and locks.

Heck, Twisted has http://twistedmatrix.com/documents/9.0.0/api/twisted.interne... I had to use it too because multiple callback chains modifying and accessing the same state had a race condition. Yeah I knew I could multiply a matrix quickly without having to acquire a lock, but I wasn't multiplying matrices I doing IO-bound things. With concurrent IO requests there is still a potential for a data race.

> So green threads may be great but they don't bring anything new to the table.

Green threads bring:

1. Lighter weight concurrency units than native threads.

2. Green threads don't fragment the library ecosystem. (For a language with batteries included this is rough). If you have been using Twisted you know what I am talking about. "Oh I found a library that does this protocol. Ah, but it is not Twisted, can't work with it. Start writing a parallel implementation

3. Provide a better abstraction without extra code bloat. When you really want to put an item in a shopping cart, do you really care anything underneath yields? You want to write : authenticate(); get_price(); get_availability(); update_cart(); respond_to_user(); and such. That code should not know about select loops and reactor and awaits. Lower level frameworks should handle that and top level code should be clean and obvious.

After switching from Twisted (even with inlineCallbacks) I cut the total lines of code in a large code base by half by using eventlet (that was before gevent), because it cut all the callbacks and handlers and all that stuff. Those are lines of code cluttering the business logic, they need maintenance, they need people to read them when bugs happen.

Are Gevent and Eventlet ideal? No, they have been always a hack. But in practice I'll take the monkey-patching vs awaits, yield or deferreds and having to hunt for or rewrite libraries which speak that particular IO "language". I understand that on paper and in small example those look neat in clean, in practice it turns into a mess.

plq on Oct 31, 2016 | root | parent | next [–]

My point was that green threads sort of emulate OS threads so should not be treated any differently. As Python already has nice library-level support dealing with such thread-based concurrency, there's just nothing to do for the core language team to support this use case.

> 2. Green threads don't fragment the library ecosystem.

They do. That's why you need to monkeypatch everything.

Thing is -- you got two ways to do IO in an async world: Use the async system calls nothing less than the *kernel* provides or use threads to use blocking system calls and emulate async IO. There's no escaping that reality irrespective of the async paradigm you are using, green threads included.

> 3. Provide a better abstraction without extra code bloat.

From what I understand, your problem has always been the GIL, not Twisted. If your business logic is not better expressed in Twisted, you should not use Twisted, period.

For some of the code I need to deal with, Twisted's callback logic fits perfectly. It makes my code more testable and easier to reason about. So that's what I'm using. For anything else, I just deferToThread and use blocking code just like normal.

This said, I'd still like to emphasize one very important point:

Here's the secret sauce of gevent: https://github.com/python-greenlet/greenlet/blob/master/plat...

A sibling comment to yours explains briefly how Windows folks have given up trying to get green threads to work even with kernel support.

I do realize the average Python programmer couldn't care less about such low level stuff. However those of us who peeked under the hood of gevent and realized how many basic assumptions it violates stays far away from it.

Green threads are the GOTO of cooperative multitasking. In case you want to switch to "structured programming" from using GOTO-based code, you need to switch to the Twisted mindset.

marcosdumay on Oct 31, 2016 | root | parent | next [–]

> Green threads are the GOTO of cooperative multitasking.

You know, in GOTO based vs. structured code, one is a mess where nobody can get things correct at the first several tries, where another is a organized piece, built observing programmers limitation.

The same does apply to bare async-io vs. green threads, but you got something missed-up there.

rdtsc on Oct 31, 2016 | root | parent | prev | next [–]

> > 2. Green threads don't fragment the library ecosystem.

They do. That's why you need to monkeypatch everything.

Not in the same way Twisted or async + yields does. Monkeypatching it not done in the library, that's the whole point. It is done in the start phase of the process once. If I get an IRC library which does uses sockets and spawns threads, that could work with Gevent, eventlet or just regular threads.

If I get a Twisted one then they returned deferreds an my main program doesn't know how to handle deferreds. Or alternatively I am using Twisted I have to find libraries which return deferreds. That's what I meant by fragmenting.

> se the async system calls nothing less than the kernel provides or use threads to use blocking system calls and emulate async IO.

It's the other way though? Green threads use async version of socket calls with a select/poll/epoll/kqueue hub (or reactor in Twisted world) but then they provide a blocking synchronous API to the higher level code.

That is usually the sanest abstractions. The only times I've seen callbacks work well is when callbacks are very short, think something like web simple web proxy for example.

In general callbacks in a complex program end up a mess from what I see. inlinedCallbacks or co-routines with yields help there, I've used those. But it is still suboptimal as library ecosystem is still fragmented and code is still cluttered with yield and awaits and so on.

> Green threads are the GOTO of cooperative multitasking. In case you want to switch to "structured programming" from using GOTO-based code, you need to switch to the Twisted mindset.

I think it is the opposite. A callback chain is an ad-hock, poorly implemented and obfuscated model of a blocking concurrency unit. That is a socket event starting a callback chain of cb1->cb2->cb3... is usually much better represented as a set of nicely blocking functions calls fun1->fun2->fun3. Except callbacks are scattered all over. And just because they are callbacks doesn't mean you don't locks and semaphores, you can still have data races between another callback chain started from a different socket which also calls cb1->cb2->cb3 before first one finished.

Also noticed that languages which are used in highly concurrent environments follow the same paradigm, namely Erlang. It is not a sequence of callback but rather isolated blocking concurrency units. Inside each unit calls are blocking but there can be many concurrent (and run in parallel) such concurrency units. Go does the same.

stygiansonic on Oct 30, 2016 | prev | next [–]

For background, the author founded the Flask project [0] among others, and contributed to many other well-known Python projects.

0. http://lucumr.pocoo.org/projects/

ubernostrum on Oct 31, 2016 | parent | next [–]

For more background, Flask embodies a lot of what Armin likes about how to build things, and there's room to disagree with him on it.

(for example, Flask attaches the request to a magic thread-local variable, while Django -- to pick the one I prefer -- requires you to explicitly pass it around and write functions to take it as an argument; there's a parallel in Armin's complaints about asyncio requiring you to explicitly pass things around instead of accessing magic thread-local storage)

forgotpwtomain on Nov 1, 2016 | root | parent | next [–]

From my understanding Armin's complaint isn't solely that you have to pass it around but the fragmentation of not having a standard way of doing it:

>> pass the event loop to all coroutines. That appears to be what a part of the community is doing. Giving a coroutine knowledge about what loop is going to schedule it makes it possible for the coroutine to learn about its task.

>> alternatively you require that the loop is bound to the thread. That also lets a coroutine learn about that. Ideally support both. Sadly the community is already torn of what to do.

Personally I'm not clear on a lot of the minority-use cases that spawns some of this complexity (e.g. when and why would you need to move event loops across threads?) and am quite out of my depth as well.

dom0 on Oct 30, 2016 | parent | prev | next [–]

Also https://www.palletsprojects.com/

Btw. does someone know why some things where moved from Pocoo to Pallets?

loulouxiv on Oct 30, 2016 | root | parent | next [–]

Armin wanted to step back a little from these projects and moved them to pallets in order to make them more community-driven

__derek__ on Oct 31, 2016 | root | parent | prev | next [–]

Here's the blog-post announcing it.[1]

[1]: https://www.palletsprojects.com/blog/hello/

anacrolix on Oct 30, 2016 | prev | next [–]

I agree with Armin that Python is becoming burdensomly complicated. Since Python 3.2, the language has started to accumulate a lot of features, Guido seems to be saying yes to everything. Furthermore, the Python 3 fiasco has severely affected Python's popularity. It would be in much greater use now if the clean-up had been done more gradually.

krick on Oct 30, 2016 | parent | next [–]

I don't quite see why it's being downvoted. Like it or not, but that's what is happening: Python's 2.4-2.7 success was largely due to it's simplicity. It more or less did correspond to values defined in `import this`. It wasn't particularly performant nor "safe/foolproof", it wasn't even that powerful as a language. It lacked (and still lacks) good error reporting system. But it was lean, easy to get started with, agile enough, expressive enough. It was always possible to make something unintelligible using reflection and magic methods, but it is easily distinguishable form "how stuff should be written" and otherwise you could be quite sure there won't be any surprises.

This is why python is still used for it's purpose. CLI utils, bots/crawlers, tf/pandas/scikit-learn, REST-APIs on top of flask.

Then hard times came. First, many year long story (still not concluded) of transition between 2 and 3. Then all this stuff. Sure, there is such thing as progress. Stuff is invented for some purpose. But now, in 2016 and at v3.6 — Python isn't what it has been loved for. Not easy-to-start-with, nor simple. This return/yield fuck up, showed in the article is absolutely huge deal, for example, and it is not about asyncio per se. Async stuff is always complicated, it wouldn't be that bad if it was all it's about.

If some 5 years ago one would use Python just because "why not, I just need to get stuff done", now it's quite likely that after struggling with all this micro-nuisances he would go with golang/js/php/whatever instead.

int_19h on Oct 30, 2016 | root | parent | next [–]

The thing about Python is that unlike most other languages, you don't *have* to deal with that complexity. You can still write Python 2.7 style code, no problem.

This approach to async, though, is just a language feature that's becoming mainstream right now. C# has it, ES7 has it, C++ has a working paper on it etc. Python actually had the benefit of watching how things work out elsewhere before implementing it all.

thijsvandien on Oct 30, 2016 | root | parent | next [–]

The idea that complexity doesn't matter if you don't use it sets Python on the path to becoming the next C++. Pick the subset of the language that you like. Then carefully watch your libraries, because if they use a different subset, you still have to deal with what's in there (and Python is a library heavy language!). It goes against everything Python stands (stood) for. Definitely not what I'm looking for.

int_19h on Oct 31, 2016 | root | parent | next [–]

This is not at all a new problem for Python. In fact, it's possibly more of a problem for Python than it is for C++, because Python's dynamic nature and exposure of many internal mechanisms makes it possible to do some pretty crazy stuff in the libraries.

For example, speaking of async - even 2.7 already has Twisted, and an ecosystem of libraries around it.

The only two ways I can see it being solved is either by making it more of a toy language (which is great if you're just writing short scripts, but it's not really what it's supposed to be about); or by having a very centralized "best practices" enforcement that basically forces libraries to conform through peer pressure, like Java - which has its own disadvantages aplenty.

vegabook on Oct 31, 2016 | root | parent | prev | next [–]

no you can't. Unicode by default is a complete pain in the butt when working with libraries that want bytes. It's just a complete mess.

int_19h on Nov 1, 2016 | root | parent | next [–]

That's because it's not *unneeded* complexity. Making you stop and think about what you're doing when you say "just treat these bytes as a string" (or vice versa) is to prevent buggy code that only works properly with Latin-1, and similar kind of thing.

If some library messes it up, and requires you to provide bytes that are semantically a string, it's a design flaw in the library.

vegabook on Nov 1, 2016 | root | parent | next [–]

okay try and use any networking library such as zmq, and see what a pain in the butt it is, most definitely refuting the idea that "you can write python 2.7 code if you want". You're going to be polluting your code with .encode .decode all over the shop. Same with xrange, dict access. All sorts of cruft has been introduced in the pursuit of "seriousness". And now we have this post (and most of the comments) saying that 3's flagship win, asyncio, is actually a dog. I reluctantly moved to 3 last year and honestly, I cannot find any wins whatsoever. Seriously considering Lua, although it also has its 5.3 v 5.2 problems not to mention Luajit "fork".

int_19h on Nov 7, 2016 | root | parent | next [–]

> You're going to be polluting your code with .encode .decode all over the shop.

Well, do those encode/decode calls serve a purpose? Does encoding matter?

If so, they're not cruft - they do something that needs to be done. You may dislike the fact that it's extra complexity, but any speaker of a language for which ASCII (or Latin-1) is not sufficient will rejoice knowing that you can't write code that breaks when we try to use it anymore. I remember how much of a hassle this kind of stuff was back in DOS/Win9x days, and even in early 00s; and I also remember how a hard push for Unicode as *the* string encoding in mainstream languages like Java and C# did a lot to rectify that.

If not, then why would you need to encode something just to decode it later, or vice versa? Why not just pass bytes/bytearray around? Again, if the library requires doing so (because they demand the data to be passed as a str specifically, even though it's never actually treated as a string), then that's a bug in said library, and you should complain to/about its authors instead.

goatlover on Oct 31, 2016 | root | parent | prev | next [–]

As If JS isn't becoming more complicated by the year, and PHP didn't add a ton of features in 5. Anyway, Python ranks in the top 4 most popular language rankings on multiple sites, and plenty of people are still learning it.

pekk on Oct 31, 2016 | parent | prev | next [–]

Python had "a lot of features" in the 2.x series. This has nothing to do with Python 3.

earthnail on Oct 30, 2016 | prev | next [–]

I just got started on Python's asyncio. It took me a long time to understand how to get things done, but after about a week of doing small example projects in the evenings, it finally 'clicked'. For the record, I never worked with gevent before, and have only done a small webserver with Tornado a long time ago, so I can't comment on how it compares to other solutions. One thing that made me wait for asyncio is the fact that, despite being experimental, it has a lot more of an "official" feeling to it.

I'm trying to stay away entirely of the Python 3.4 way with coroutine decorators, and am using only await and async in Python 3.5. The async code I wrote has to live in parallel with regular synchronous Python code in a large scientific code base, but migrating our custom database adapter to an asynchronous codebase without breaking old synchronous code was surprisingly easy.

Debugging is, in my opinion, a pain. Stacktraces can be extremely long and very hard to understand. The only profiler that seemed to give useful results was pprofile (in sampling mode). I also still don't fully understand why there's both Futures and Tasks - I probably didn't spend enough time understanding the difference, but that just means the author of this blog post is right. Mixing asyncio with threads and/or processes, however, is surprisingly easy and elegant.

I hope the Python developers will have the courage to break backward compatibility in the asyncio module, and will remove the old yield from and @coroutine way of doing things. That would probably help a lot in reducing confusion. There's still not a lot of information about asyncio when you google for it, so the amount of existing code and examples that that would invalidate would not be too high.

All in all, we are very happy with asyncio. We use it mainly to add concurrency to small sections of our code base. By default, all our code is synchronous, with some heavy I/O-bound functions exposing async versions, too. asyncio allows us to

parallelise these sections without the use of a thread pool, and thanks to Futures/Tasks and queues, it's actually very easy to do this in a "streaming" fashion if the order of processing of the outcome of your concurrent tasks matters. Add to that the executors which allow you to run stuff in sub-processes when you're CPU-bound (instead of I/O), and it makes for a fairly solid tool.

takeda on Oct 31, 2016 | parent | next [–]

> I also still don't fully understand why there's both Futures and Tasks - I probably didn't spend enough time understanding the difference, but that just means the author of this blog post is right.

Because they are essentially the same.

Task is just extending Future and adds extra functionality (like for example keeping track of tasks schedule in given event loop).

It is created when you call ensure_future() or loop.create_task(). You are not supposed to create it directly, so if you're wondering whether you should use Future or Task you should use Future.

lilbobbytables on Oct 31, 2016 | parent | prev | next [–]

I learn best by playing with small projects and implementing, and I've been wanting to wrap my head around asyncio for a while now.

Would you happen to have some code examples, or just examples of what kind of sample projects you built?

doh on Oct 30, 2016 | prev | next [–]

So I'm not the only one. I wrote larger-ish project while using asyncio and it is pain. The syntax is very unfamiliar (especially in 3.5 with async/await), the documentation is confusing and it's very hard to debug it in general. Also it's very hard to combine/stack multiple IO heavy events (make 5 calls to these URLs and whichever is done and returns these task run them in parallel).

wumpus on Oct 30, 2016 | parent | next [–]

That last part is exactly what my webcrawler does with asyncio plus threads: https://github.com/cocrawler/cocrawler

This is the part that sends work to a thread: https://github.com/cocrawler/cocrawler/blob/master/cocrawler...

I agree that this was confusing in the docs. Docs can be improved. It really helped that this isn't my first crawler written using cooperative multitasking.

dom0 on Oct 30, 2016 | parent | prev | next [–]

I feel you.

Having worked with it a bit more I'd say that these things will work out as time goes on. Currently there is quite a bit of fragmentation of async-solutions in Python that are often somewhat-but-not-fully compatible; the docs need a lot of love as well. While the reference in itself is okay, cross-references and conceptual docs (the latter being extremely important IMHO) are lackluster or non-existent.

Also there seems to be a lot of confusion caused by the lack of docs around the difference of the keywords and the "asyncio" module (and it's various other forms). The former is just a coroutine / suspension engine and has relatively little to do with asyncio.

Specifically your last request: https://docs.python.org/3/library/asyncio-task.html#asyncio....

It is inherently somewhat limited by requiring the same underlying loop. But this is hard to change on a principal level...

In a broader sense, I like the increased visibility this gives to async APIs; with the "old ways" this could be easily overlooked. Since good API design is even more important in an asynch piece of software than in a sync piece of software I feel like this is quite an advantage, avoiding bugs and highlighting API issues directly.

With the old way, for example, it was relatively easy to confuse a synchronous method and a coroutine, since it's completely silent. You'd only notice this when stuff doesn't happen that was supposed to happen, and you can only see the bug at the call site if you know whether it's a method or a coroutine. This can't happen with the new keywords anymore. I think this is arguably their greatest advantage.

lyschoening on Oct 30, 2016 | prev | next [–]

There are a few new concepts that need to be learned when beginning work with asyncio as well as some confusing naming choices (who would expect that a coroutine is not the same as a coroutine function?). Some of the tooling is also lacking. Armin points out, no doubt with a look at server frameworks, that there is no elegant way to access the context of a task.

That being said, as someone who started working with asyncio in Python 3.5 none of it feels particularly difficult to understand. Asyncio needs more work, sure, but the API so far is relatively straightforward.

The overloading of iterators/generators is a bit odd. But the same approach that was taken with JavaScript -- and it's nothing anyone working with Python 3.5 or above will be exposed to. If I recall correctly, Python 3.6 will even feature async generators. Any developers diving into this using that release won't have to ask themselves why they can't 'yield' from within an 'async' function.

the_mitsuhiko on Oct 30, 2016 | parent | next [–]

I don't find it hard to write asyncio code but I find it hard to generically interface with other people's asyncio code and that is lacking basic support like a context object or logical call contexts or just the lack of agreed upon usage patterns.

It does not help that asyncio evolves in the stdlib and changes with every major Python version. It might be less of an issue if this was pip installable I suppose. Right now writing utility code for asyncio is targeting many things.

sitkack on Oct 31, 2016 | root | parent | next [–]

I was talking with David Beazley and he had some of the same confusions around asyncio. I think it would be nice to revisit it with a round table and fix it in 3.7.

webmaven on Oct 31, 2016 | root | parent | next [–]

This should definitely happen, if at all possible.

nerdwaller on Oct 30, 2016 | root | parent | prev | next [–]

Definitely fair, as time progresses I think cleaner patterns will emerge. By time many of the major libraries were around, there was a semi-standard Python programming style that was fairly easy to adhere to. Since this is somewhat of a paradigm shift for the stdlib it will take a little time too.

Personally I have been looking at some of the patterns taken by the team on aiohttp - I think they've done a good job and, iirc, one of their members is a core contributor to Python.

stuaxo on Oct 30, 2016 | parent | prev | next [–]

I started on py3.5 - but haven't been able to use the new syntax as I'm using aiozmq so had to use @coroutine.function.

The whole experience has been mega confusing as there are so many similar yet slightly incompatible concepts.

1st1 on Oct 30, 2016 | root | parent | next [–]

You can call old style coroutines (generators wrapped in asyncio.coroutine) from async/await code directly.

```
@coroutine
def foo(): return 123

async def bar():
    await foo()
```

erikb on Oct 30, 2016 | prev | next [–]

From what I learned through youtube videos and meetups it seems to solve a problem that doesn't exist (getting all these patterns into python) and in return doesn't solve the problem people hope it would solve (multiprocessing made easy and pythonic). That's why people who want to pretend to be smart (like me a few years back) find this totally attractive, king's new clothes style. Nobody really understands it so they can act like they really do something meaningful with it.

How do I get to this painful conclusion? Well, as I said, I was (and probably still am to some degree) just like that. And to me it looks nearly irresistably interesting. But at the same time I also don't know what I would use it for, what it would actually improve for me. And since the need to pay my rent forced me to use my time more practically I didn't get around to looking at asyncio more in depth. Both these things together make me believe it's not really solving a real problem.

pekk on Oct 31, 2016 | parent | next [–]

The name "asyncio" includes the substring "io" and is clearly about asynchronous I/O, so it's hard to see why anyone would think its purpose was "multiprocessing made easy."

scribu on Oct 31, 2016 | parent | prev | next [–]

Not sure what you mean by "getting all these patterns into Python".

Just because you don't work on projects that can benefit from concurrent IO doesn't mean they don't exist. I work on such a project.

kamyarg on Oct 30, 2016 | prev | next [–]

Python core developer Brett Cannon wrote a nice(and long) post about understanding it a couple of months ago[1], that might help. After reading that I felt like I could use it, but never got the chance to try.

[1] http://www.snarky.ca/how-the-heck-does-async-await-work-in-p...

orf on Oct 30, 2016 | prev | next [–]

I'm working on some Asyncio stuff right now. Asynchronous programming seems pretty natural to me, but other people do struggle to wrap their heads around somewhat confusing terminology: tasks, co-routines, awaitables, event loops. Underneath the terminology the theory is pretty simple.

And boy is it powerful. If you ever find yourself doing network requests in a loop (for url in list: requests.get(url)) then a small bit of refactoring and a sprinkling of asyncio will speed this up immensely.

But it's not just for network calls, you can `await` on threads and processes. It's a joy and I think it's one of the best things in Python right now.

> sidlls on Oct 30, 2016 | parent | next [–]
>
> Have you tried comparing the performance of asyncio based network requests versus multithreaded requests? And also compared the relative complexity of the code?
>
> I have never used asyncio in Python, mainly because the very use case you described is solved with multitheading, but that doesn't mean it's solved best that way of course.
>
>> orf on Oct 30, 2016 | root | parent | next [–]
>>
>> > Have you tried comparing the performance of asyncio based network requests versus multithreaded requests
>>
>> Nope, but the performance of an individual request overhead isn't much of a data point. The advantage is that it scales to thousands of connections easily, there are no concurrency/threading problems, and you can mix and match protocols easily. None of that is easy with threading. Threads can also be quite expensive to start and manage.
>>
>> That specific case is handled by threading, yes, but if you're making a webservice that makes requests to a bunch of endpoints during when processing a HTTP request and also sends output to IRC/Slack whilst simultaneously serving files over FTP and launching a bunch of external processes for good measure then asyncio has your back.
>>
>> grok2 on Oct 30, 2016 | root | parent | prev | next [–]
>>
>> The problem with multi-threading is knowing how to structure your program correctly to avoid issues with concurrent data-structure accesses, etc -- it requires more careful coding. With asyncio, you don't have to think about these issues.
>>
>>> sidlls on Oct 30, 2016 | root | parent | next [–]
>>>
>>> I...disagree. And I'll leave it at that.
>>
>> Animats on Oct 30, 2016 | root | parent | prev | next [–]
>>
>> Python's multithreading is insanely inefficient, because of the Guido von Rossum Memorial Boat Anchor. Anything in Python can mess with the innards of anything else at any time, including stuff in other threads. (See "setattr()"). There's no such thing as thread-local data in Python. This implies locking on everything. CPython has one big lock, the infamous Global Interpreter Lock. Some other implementations have more fine-grained locks, but still spend too much time locking and unlocking things. One Python program can thus use at most one CPU, no matter how many threads it has.
>>
>> This basic problem has led to a pile of workarounds. First was "multiprocessing", which is a way to call subprocesses in a reasonably convenient fashion. A subprocess has far more overhead than a thread; it has its own Python interpreter (some code may be shared, but the data isn't) and a copy of all the compiled Python code. Launching a subprocess is expensive. So it's not a good way to handle, say, 10,000 remote connections.
>>
>> Now there's "asyncio", which is the descendant of "Twisted Python". That was mostly used as a way for one Python instance to service many low-traffic network connections. The new "asyncio" is apparently more general, but hammering it into the language seems to have created a mess.
>>
>> After the Python 3.x debacle, which essentially forked the language, we don't need this.
>>
>>> the_mitsuhiko on Oct 30, 2016 | root | parent | next [–]
>>>
>>> > There's no such thing as thread-local data in Python.
>>>
>>> There is. threading.local in all aspects is thread local data.
>>>
>>> > Now there's "asyncio", which is the descendant of "Twisted Python". That was mostly used as a way for one Python instance to service many low-traffic network connections. The new "asyncio" is apparently more general, but hammering it into the language seems to have created a mess.
>>>
>>> I think the mess was created before 3.5. Had the whole thing started out with the async keywords we might have been spared `yield from` which is a beast in itself and a lot of the hacky machinery for legacy coroutines. I do think however we can still undo that damage.
>>>>
>>>> Animats on Oct 30, 2016 | root | parent | next [–]

threading.local in all aspects is thread local data.

You can still pass data attached to threading.local to another thread. Another thread may be able to get at threading.local data with setattr(). There's no isolation, so all the locking is still needed.

This is a hard problem. There's real thread-local data in C and C++, but it's not safe. If you pass a pointer to something on the stack to another thread, the address is invalid and the thread will probably crash trying to access it. C++ tries to prevent you from creating a reference to the stack, but the protection isn't airtight. In Rust, the compiler knows what's thread-local, as a consequence of the ownership system. Go kind of punts; data can be shared between coroutines, but the memory allocation system is mostly thread-safe. Mostly. Go's dicts are not thread-safe, and there's an exploit involving slice descriptor race conditions.

the_mitsuhiko on Oct 30, 2016 | root | parent | next [–]

> You can still pass data attached to threading.local to another thread.

You can in most languages. Only rust I know has enough information to prevent that.

_pgmf on Oct 30, 2016 | root | parent | prev | next [–]

On what are you basing your statement that multithreading in Python is insanely inefficient? Despite the fact that the GIL prevents multiple threads from running in parallel, using multiple threads can give you a huge boost if IO is your bottleneck.

I think that it's irresponsible to make a blanket statement like this, because there are many use-cases for multiple threads in Python. Sure, one of the obvious ones (parallel processing) doesn't work, but besides that threads can be extremely useful.

I'm also unclear on what you mean by "no such thing as thread-local data" when there is `threading.local()` that does exactly that.

Lastly, I don't think multiprocessing was created as a workaround for threading per-se. Rather it was a workaround for the global interpreter lock.

sidlls on Oct 31, 2016 | root | parent | next [–]

And in either case my view is concurrency is best done in a language where there is proper support for it, whether the model is threaded or processor based concurrency.

sidlls on Oct 30, 2016 | root | parent | prev | next [–]

Not that I disagree with (m)any of the points you raise, but with due respect I think it's a bit off-topic from my comment.

Were it my choice, any time the need for concurrency comes up at my job I'd prefer to use a statically typed, compiled langauge like C++ or Java (or, once I've familiarized myself with Rust's implementations, that language), and this kind of discussion wouldn't even come up. I like python as a rapid-prototyping language. For the kinds of numerical computations and data-laden I/O bound work I do I find it sorely lacking, and consider it an unfortunate choice for production work.

feisuzhu on Oct 30, 2016 | parent | prev | next [–]

```python from gevent import monkey monkey.patch_all()

for url in list: gevent.spawn(lambda: requests.get(url)) ```

I concern this a problem already solved by gevent(or Erlang process / goroutine). Actually, I didn't see a benefit introduced by asyncio. Monkey patching seems scary but it works quite well in real projects. At least in my medium sized (50KLOC) game project.

orf on Oct 30, 2016 | root | parent | next [–]

```
responses = await asyncio.gather(*(aiohttp.get(url) for url in urls))
```

Gevent isn't available everywhere, and it has compatibility issues. Monkey patching is also... horrible.

Spiritus on Oct 31, 2016 | root | parent | prev | next [–]

Don't use lambda.

```
gevent.spawn(requests.get, url)
```

marcosdumay on Oct 31, 2016 | parent | prev | next [–]

Handling network requests is the textbook example for green threads (mostly because it fits in a paragraph), and if you never tried it you should step outside of the Python ecosystem for a short while to get a clear picture.
```

Tasks that require bare async programming are incredibly rare. There's a small set of patterns that covers almost every application ever written, but it's not completely covered by Twister (or at least didn't use to be, there's a few years that I don't touch it).

Honestly, there's no problem with Python exposing the low level async primitives. That's good, and very pythonic. My only problem is that everybody is talking like this is the complete package and Python is now a good choice for async programming.

balloob on Oct 30, 2016 | prev | next [–]

At home-assistant.io we just migrated our core from being based on threads + locks to use asyncio. We managed to keep a full backwards compatible API available so we can slowly migrate other parts of the system over to async.

Asyncio has a steep initial learning curve (especially in our hybrid setup) but it's well worth it. We target low resource computers like Raspberry Pi and using async over threads has speed up things a lot.

The biggest catch is that while writing code you have to think about every function that you use. Is it doing I/O, is it a coroutine or is it callback/async friendly.

ddorian43 on Oct 30, 2016 | parent | next [–]

... why do you need locks in threads and not in asyncio ?

paulddraper on Oct 30, 2016 | root | parent | next [–]

Because with asyncio you can run on one thread.

Before, you needed multiple threads, or else incur high I/O costs. But then you have to manage your shared state across threads, hence locks.

Their code is now probably async single threaded code, like JS, where locks are irrelevant.

ddorian43 on Oct 30, 2016 | root | parent | next [–]

But don't threads run cpu-stuff in 1 thread, by not switching? (in python) The same as async but more overhead/slower.

paulddraper on Oct 30, 2016 | root | parent | next [–]

Only one thread runs at a time because of the GIL, but the threads can preempt each other wherever between opcodes (http://effbot.org/zone/thread-synchronization.htm#atomic-ope...)

Imagine two threads running

```
a += 1
```

That's four opcodes. Only one operation will run at a time, but one thread might preempt the other midway through the four operations, and `a` will have been incremented by 1, not two.

In some sense, mulithreading with Python's GIL is the worst of both worlds: you can use only one CPU, but you still need to account for concurrent state mutation. (Of course, it doesn't have the function color problem of asyncio http://journal.stuffwithstuff.com/2015/02/01/what-color-is-y...)

balloob on Oct 30, 2016 | root | parent | prev | next [–]

We no longer need locks in the core because there is now only 1 thread that interacts with the core. We still have other threads but all they do is schedule tasks to be run on the event loop.

https://github.com/home-assistant/home-assistant/blob/dev/ho...

takeda on Oct 31, 2016 | root | parent | next [–]

There's still a possibility that a lock might be needed if you use await/yield from inside critical section (another co-routine could make changes at that point). If you don't yield then indeed locks are not needed.

hamiltonkibbe on Oct 30, 2016 | root | parent | prev | next [–]

Probably because you don't have to worry about context switches in the middle of operations, since they happen at 'await' or 'yield's in coroutines, rather than whenever the OS task timer interrupts

the_mitsuhiko on Oct 30, 2016 | root | parent | prev | next [–]

You need locks in asyncio as well.

xapata on Oct 30, 2016 | root | parent | next [–]

Only if you have multiple threads/processes/coroutines sharing resources.

DasIch on Oct 30, 2016 | root | parent | next [–]

"Only"? You need locks in exactly the same scenarios as you do when you use multiple threads.

xapata on Oct 30, 2016 | root | parent | next [–]

Good point. I should have said that asyncio allows patterns that use fewer shared resources.

Actually... I take that back. Asyncio is solving a different problem than the locking of shared resources.

ddorian43 on Oct 30, 2016 | root | parent | prev | next [–]

I know you do, but I asked for his implementation (which needed locks on threads but not on async)

yladiz on Oct 30, 2016 | prev | next [–]

Maybe I'm not the intended audience for this article, but I don't really have a problem with a lot of the warts of asyncio. It really is pretty simple to use if you don't have your mind hard set to a different way of thinking. It wasn't as simple in Python 3.4, but then the async/await keywords became part of the language in Python 3.5 and simplified things. Understanding asyncio took me a few days to understand the complexities: the event loop usage and how this works with async/await, how an async function is different than a normal function, and a few other things. Once I understood those complexities (which I don't mind because it is a new part of the language) it was easy. Granted, I feel like it's easier to just work around issues rather than complain about them but I didn't find particularly many issues about asyncio.

As an aside, not to pick on Armin but he was also complaining about Python 3 about things that may have been valid, but were more that he didn't like the way that Python 3 worked because he liked Python 2 more, and hid that fact by writing lengthy blog articles about how he doesn't like certain Python 3 things. I do find it a little strange that he does complain about these things publicly rather than trying to fix the things he doesn't like (if they are fixable), especially given his reputation in the community and his Python projects like Flask, because it makes him seem whiny and solves none of the issues that he's presenting.

the_mitsuhiko on Oct 30, 2016 | parent | next [–]

> I do find it a little strange that he does complain about these things publicly rather than trying to fix the things he doesn't like (if they are fixable), especially given his reputation in the community and his Python projects like Flask, because it makes him seem whiny and solves none of the issues that he's presenting.

It's easy write code, it's harder to write specs and design systems and it's hardest to convince others. I'm very bad at the last part. My only real attempt to improve python 3 that went anywhere was to get the u prefix back. My suggestions for bytestrings were not very popular for instance.

yladiz on Oct 31, 2016 | root | parent | next [–]

Okay, fair enough, it's harder to think about a problem than write the solution to the problem in general, at least for software development. But it would be better if you did try to talk to the Python developers in a constructive manner and try to get whatever you feel is wrong with Python fixed. Writing blog posts is fine, and I'm not saying that voicing your opinion is bad, but there is a threshold between voicing an opinion and actually doing something about it, and while I feel you do contribute to the Python community, you don't contribute to Python itself even if you have the ability to and after years of blog posts you've crossed that threshold. The reasons that you don't attempt to contribute I'm not sure -- lack of patience for the process, unable to convince others, designing a system that fixes the issues you're finding. The biggest thing is that if you want something fixed and have the ability to fix it (which I feel you would have) don't just sit around and complain -- do something about it. The best way to convince a developer is to write code to prove your point.

jdnier on Oct 30, 2016 | prev | next [–]

The post mentions in passing David Beazley's curio[1] project. Is anyone using curio for async programming instead of asyncio?

The curio docs are fun to read through and didn't leave me feeling lost. They're also full of understandable examples of using the new async/await syntax.

I've made some simple scripts with curio but have found I keep hesitating take on the asyncio docs to learn it "the real way". Any thoughts on whether curio might be a plausible alternative to asyncio?

[1] http://curio.readthedocs.io/en/latest/tutorial.html

scribu on Oct 31, 2016 | parent | next [–]

Since it's polling for I/O events and doesn't use threads [1], it does indeed sound like a plausible alternative for doing non-blocking I/O.

I think I'm going to take a stab at using it, since asyncio doesn't have a mature ecosystem around it anyway.

[1] http://curio.readthedocs.io/en/latest/index.html#under-the-c...

r4pha on Oct 30, 2016 | prev | next [–]

I had a similar experience with python 3's asyncio. I have worked with gevent, which has an arguably less "elegant" interface (with monkey patching, for instance), but with which it is so much easier to be productive. I was frustated for having

dificulties understanding and using asyncio. The author is a better python programmer than I am, so I supposed there is really a problem there.

mangeletti on Oct 30, 2016 | prev | next [–]

The one thing that I think is absolutely lovely is the syntax:

```
async def spam(eggs):
    ...
```

It can't get much better than that.

My hope is that the implementation will become:

A) simpler

B) more optimized

C) out of the box functional (e.g., no need to manage event loops and other things yourself)

D) unified (e.g., the zen of python even states that there should be one way to do something)

I also strongly agree with the comment @RodericDay made herein, pertaining the new typing syntax and the addition of yet another string formatting syntax (a dangerously unexplicit one).

> voltagex_ on Oct 30, 2016 | parent | next [–]
>
> Have a look at the way C# does it. I think it comes pretty close.

denfromufa on Oct 31, 2016 | prev | next [–]

I find that Armin's blog posts about Python 3 are generally destructive, not constructive. A constructive way would be to post suggestions on python-ideas and python-dev mailing lists or report bugs and feature requests on issue tracker.

Python is open-source project and there is not a single core developer working full-time on the language unlike his beloved Rust.

> raymondh on Oct 31, 2016 | parent | next [–]
>
> I've always found Armin's posts to be intellectually honest and thought provoking. And unlike your ad-hominem comment, he itemizes his concerns and provides details (such as the performance comparison with David Beazley's curio project).
>
> At the very least, it is a warning sign that a notable and highly experienced Python expert is having a hard time grappling with the best practices (or even workable practices) for a significant new feature set: "I know at least that I don't understand asyncio enough to feel confident about giving people advice about how to structure code for it."
>
> As far as I can tell, not a single respondent to this thread has indicated that to the contrary, they have been able to say that they do feel confident enough to give people advice on how to structure code with asyncio.
>
> At the very least, that means that we have a documentation and communication problem which is either intrinsic to the new API or something that will work itself out over the next few years.
>
>> ubernostrum on Oct 31, 2016 | root | parent | next [–]
>>
>> For what it's worth, I read Armin's critiques, but I take them with a grain of salt; it's clear that what he wants from a language, and what other people want from Python, diverged a while back and are probably irreconcilable at this point. That doesn't mean he doesn't have good points, but does mean that I read his articles through a lens of "the language he really wants probably is never going to be Python again".
>>
>>> the_mitsuhiko on Oct 31, 2016 | root | parent | next [–]
>>>
>>> > I read his articles through a lens of "the language he really wants probably is never going to be Python again"
>>>
>>> I always had that opinion that what I would like Python to be like is never going to happen. This is not something new with Python 3.

jftuga on Oct 30, 2016 | prev | next [–]

I wrote a fast tcp port scanner using Python 3.5 and concurrent.futures.ThreadPoolExecutor.

See lines 123-124:

https://github.com/jftuga/universe/blob/master/tcpscan.py

I have used this under Linux, OSX and Windows. It's cool to add the Thread Count field in Task Manager and then see something I wrote use so many threads! I am more of a sys admin, so this code could be better - but it seems to work very well. :-)

> hueving on Oct 30, 2016 | parent | next [–]

quick github tip: if you click the line number on the left you can get a link directly to the line you are referring to.

extra tip: after clicking the line, press 'y' on your keyboard and you'll get a link to the file in it's state at the current commit so future commits won't break your old hyperlinks.

jftuga on Oct 30, 2016 | root | parent | next [–]

cool, thanks

orf on Oct 30, 2016 | parent | prev | next [–]

Whatever works for you, but changing 20 lines of code in this to use asyncio will send your throughput through the roof.

revelation on Oct 30, 2016 | root | parent | next [–]

The limiting factor for a fast TCP scanner is not your favorite async toolkit.

orf on Oct 30, 2016 | root | parent | next [–]

Well, no, of course not. Don't be stupid. The default ThreadPoolExecutor size is the number of cores you have on your laptop. In my case, 8. So that's 8 concurrent connections at once. That's, frankly, shit.

Use asyncio (or roll your own toolkit, or just spawn 'nmap') and you get 20000 concurrent connections.

Which is better?

rantanplan on Oct 30, 2016 | root | parent | next [–]

You're conflating concurrency with parallelism.

Your 20000 concurrent connections with asyncio are not parallel. You can have 20000 threads if you want.

fzzzy on Nov 1, 2016 | root | parent | next [–]

Are you sure about that? Have you ever tried starting 20,000 threads? Try it.

orf on Oct 30, 2016 | root | parent | prev | next [–]

The connections are actually handled by the OS in parallel, it's the callbacks to Python that are not. Why do you need a whole OS thread to handle sending a SYN/ACK?

As you well know 20000 threads is not a great way to do anything. Especially in Python.

the_mitsuhiko on Oct 30, 2016 | root | parent | next [–]

I agree that threads in Python are not great but 20000 threads is otherwise not an issue.

fzzzy on Nov 1, 2016 | root | parent | next [–]

Are you serious? Have you ever tried starting 20k threads?

njharman on Oct 30, 2016 | root | parent | prev | next [–]

Not using something to deserialize all that blocking I/O waiting on the internet is THE limiting factor.

adolgert on Oct 30, 2016 | prev | next [–]

The Fluent Python book has a nice set of chapters on coroutines, futures, and async.io. They present not the whole of what it does but one way to do it, which helps.

scardine on Oct 30, 2016 | prev | next [–]

Armin criticism of Python 3 is nothing new.

He is the author of several popular projects including the web framework "Flask". This makes him a person very respected in the Python community - personally I love his taste for interface design.

I wish he could interact better with the core team, because some of his rants are not as constructive as they might be.

_pgmf on Oct 30, 2016 | parent | next [–]

Whoa... Armin has never, to my recollection, been malicious in his criticisms of Python 3. He simply disagrees with some of the core team's decisions. And he's far from the only one.

It's not Armin's duty to interact in any way with the core team. Hell, the core team should be working to please Armin if you ask me. He's your target user, and he's disappointed with your product, it's not his fault.

I think Armin does a wonderful job providing a voice for those of us who are increasingly disenchanted with Python 3.

solipsism on Oct 31, 2016 | root | parent | next [–]

We know it's you Armin!

whyever on Oct 30, 2016 | parent | prev | next [–]

> I wish he could interact better with the core team, because some of his rants are not as constructive as they might be.

Can you be more specific? How could it be more constructive?

scardine on Oct 30, 2016 | root | parent | next [–]

Simplicity is a core value of the Python community; this kind of change goes through a long discussion before getting implemented, so I wonder how we ended up with such convoluted design (the reasoning is documented in the PEP system but most of it is beyond my wizardry level).

Perhaps this is me showing some cultural bias (I'm Brazilian), but in my social context it is considered rude to make this kind of criticism in public unless you tried your best to address the problem in a more restricted exchange. The more authority you have, more you are expected to come with a better proposal instead of just pointing out the mess.

the_mitsuhiko on Oct 30, 2016 | root | parent | next [–]

asyncio is not my construction site and I do not intend to make it mine. I am not an expert in anything asynchronous and as such will not try to make myself that person.

This entire adventure started because I was looking for a way to get logical call contexts working in it and this shows that there is not enough internal machinery to support that at the moment.

I'm not going to fight for that however. I hope my post was rude.

rch on Oct 30, 2016 | root | parent | next [–]

Thanks for taking the time to write out your concrete experiences and qualitative impressions in such detail. It's really a tremendous service and I'm sure it'll be well received.

the_mitsuhiko on Oct 30, 2016 | root | parent | next [–]

For what it's worth I would like to do a PEP on accessing event loops from coroutines and logical call contexts but having read some of the conversations that already took place to improve asyncio I am less thrilled about being involved.

1st1 on Oct 30, 2016 | root | parent | next [–]

I suggest to proceed with the PEP if you have a good idea about the API. Get it out there, it's not required to spend hours on mailing lists - other people can do that if they like it.

whyever on Oct 30, 2016 | root | parent | prev | next [–]

> unless you tried your best to address the problem in a more restricted exchange.

That sounds like orders of magnitude more work. In case you are not interested in performing that work, it seems more constructive to me to share your thoughts rather than to say nothing.

991821911 on Oct 30, 2016 | prev | next [–]

I still prefer Twisted, it feels more natural to me. But this is something that one cannot say aloud in many Python circles.

rgacote on Oct 30, 2016 | parent | next [–]

Long-time Twisted user (and lover) here. Lots of what we build are fairly web-transaction data translators (http to http or http to db APIs). I find that async.io, by avoiding callbacks, allows us to create a more classical-looking code structure. It works for us.

That being said: a) the Twisted environment is extremely robust and battle tested. b) I know that after six months of writing async.io code I'm still just tipping my toe into the capabilities and have yet to fully wrap my head around the details. Would love to see one of those single-topic O'Reilly books take a deep dive into async.io details.

dbcurtis on Oct 30, 2016 | parent | prev | next [–]

Pffft. Twisted core devs have embraced asyncio and have made Twisted play nice with asyncio. Do what makes sense.

bogomipz on Oct 30, 2016 | parent | prev | next [–]

>"But this is something that one cannot say aloud in many Python circles"

Can you explain why this is? What's the central issue?

X86BSD on Oct 30, 2016 | root | parent | next [–]

Twisted ROCKED. I remember when it came out everyone I knew thought it was pretty kick ass, at least those who developed in Python.

Did something happen that it's now fallen out of favor?

rspeer on Oct 30, 2016 | root | parent | next [–]

By now, people expect libraries to have thorough documentation on the Web, not just an O'Reilly book.

The fact that the port of Twisted to Python 3 is slow-going, and far from complete, also gives the suggestion that there are corners of the code that developers don't even understand anymore.

the_mitsuhiko on Oct 30, 2016 | root | parent | prev | next [–]

Twisted is still popular. Problem is that in 2.x the async ecosystems do not work together and many things might not exist everywhere. Twisted seems to have lost a bit of ground with newer things like Kafka etc.

omginternets on Oct 30, 2016 | prev | next [–]

What's missing from the asyncio docs, IMHO, is a section on common asyncio idioms and patterns. I understand the pieces individually, but I struggle to get a more global, Gestalt understanding of the system.

the_mitsuhiko on Oct 30, 2016 | parent | next [–]

The problem there is that not everybody who develops asyncjo/tulip agrees on the patterns.

omginternets on Oct 31, 2016 | root | parent | next [–]

You're probably right! I just (for once!) wish a few opinionated people would tell me what they think!

coldtea on Oct 30, 2016 | prev | next [–]

>Since I'm not clever enough to actually propose anything better I just figured I share my thoughts about what confuses me instead so that others might be able to use that in some capacity to understand it.

Coming from a well known Python developer, this gives a somewhat passive-aggressive vibe (which might not be meant at all, just sayin').

poletopole on Oct 30, 2016 | prev | next [–]

I didn't need to understand everything about asyncio to use it successfully. It's definitely a lifesaver when it comes to making lots of requests; hours of waiting turn into seconds.

sametmax on Oct 30, 2016 | prev | next [–]

Actually using asyncio tools is quite easy and straighforward. However, writting an asyncio lib, or god saves you, an asyncio framework, is really, really hard.

shad42 on Oct 30, 2016 | prev | next [–]

The complexity is the reason why I came back to gevent, especially since it now supports Python 3...

Gevent monkey patching isn't perfect but it works and gets you closer to how an event loop should be used with standard libs IMO, closer to Go.

marmaduke on Oct 31, 2016 | prev | next [–]

No one has to use this stuff if they don't want to.

Another complexity in Python is metaclasses. I've written meta classes which generate data descriptors, and was greatful that that was there when I needed it, but I also needed to look at the data model reference constantly and wrote 200% coverage tests.

carapace on Oct 30, 2016 | prev | next [–]

Coroutines are goto.

That's not a joke or anything, the semantics of coroutines are the semantics of goto statements. All this async business is just the old spaghetti sneaking back in while people are distracted by the nomenclature.

int_19h on Oct 30, 2016 | parent | next [–]

Virtual method dispatch is also a goto (worse yet, it's a computed goto!).

1st1 on Oct 30, 2016 | parent | prev | next [–]

Then for-loops are goto as well.

y11 on Oct 30, 2016 | root | parent | next [–]

A for loop is localized. I've seen Python code bases that use so many interdependent *iterators* that the code is non-refactorable and the developers just pray that it does the right thing.

That's plain iterators. Now people throw in coroutines.

xapata on Oct 30, 2016 | parent | prev | next [–]

An exception is a goto... Your conclusion doesn't follow from your premise.

carapace on Oct 30, 2016 | root | parent | next [–]

> An exception is a goto

Not quite.

> Your conclusion doesn't follow from your premise.

What conclusion?

The difference between a coroutine and a subroutine is that the subroutine cannot select the destination of a return statement: control always "returns" to the caller. Coroutines can choose where the control flow goes after they run. This is exactly what GOTO statements permit.

There is *no formal difference* between allowing coroutines and allowing goto statements.

(And we all know what goto is considered, don't we?)

Exceptions are not the same: the control is passed to a handler but the raiser/thrower (in general) does not choose the catcher.

int_19h on Oct 30, 2016 | root | parent | next [–]

> And we all know what goto is considered, don't we?

A very powerful and occasionally useful tool that's often misused, and misunderstood because of the aforementioned misuse?

carapace on Oct 30, 2016 | root | parent | next [–]

Touché.

But that's still an argument *against* smearing async all over everything like it's Nutella, eh?

;-)

int_19h on Oct 31, 2016 | root | parent | next [–]

Well, I don't really think that async/await is goto. For one thing, it cannot really be reduced to that - it's rather a syntactic sugar for CPS.

Now, CPS is *like* goto in that it's also a very powerful and occasionally useful tool, that's prone to making a mess when used improperly. But async/await fixes that exact problem - it lets you get the benefits of CPS without most of its disadvantages in terms of code readability, messiness, and ease of mistake.

So it's really more like what structured programming (loops etc) were to goto + conditionals back in the day - syntactic sugar that enforces some structure to avoid the mess that's otherwise so easy to make.

So I don't really have much problem with smearing it all over anything. It does solve a very real problem, and it seems to be the most pragmatic available choice that solves that problem (more so than, say, green threads).

carapace on Oct 31, 2016 | root | parent | next [–]

I'm actually a little puzzled by the whole async/await/coroutines thing (in Python 3.) Either the terminology is confused or they are mixing together coroutines/CPS with asynchrony, and either way I'm left doubtful.

I've written code in the past using threads, and using Twisted's Deferred et. al., etc., I know that sometimes you have a problem that *really requires* this sort of thing. My issue isn't that it's never useful, rather that I don't think Python benefits from adding this to the language when we already have it as libraries.

int_19h on Nov 1, 2016 | root | parent | next [–]

CPS *is* one approach to asynchrony, since you can interleave continuations when scheduling them in the event loop. And that's exactly what this thing

does.

As to why it's better as part of the core library - I'd say the biggest benefit is that there's a standard API for a future/task abstraction. This way, any async library is composable with any other library (in theory; there are still some warts that make it harder than it should be, some of which are described in this article).

carapace on Nov 2, 2016 | root | parent | next [–]

Interleaving continuations is cooperative multitasking; Async is about non-determinacy: you have more than one process/job/thread/core/socket/&c. and you don't know when they will run or need servicing. (I.e. select().)

Broadly speaking async processes are *not* composable. We have "theories" like Communicating Sequential Processes, and things like the SPIN model checker, but a bunch of plug-and-play libraries that just work is a pipe dream, I'm afraid.

int_19h on Nov 7, 2016 | root | parent | next [–]

I think it's a terminological ambiguity. Most times, when I hear people speak about "async" today (or speak about it myself) what they really mean is "non-blocking". And such use has been common for a very long time now - for example, in .NET 1.0 (2001), the standard pattern for non-blocking operations has been to for the function return an object implementing IAsyncResult.

xapata on Oct 30, 2016 | root | parent | prev | next [–]

The coroutine can only choose to jump into a routine designed for that purpose, not an arbitrary line of code. It's almost a goto in a similar way to exceptions being almost a goto.

carapace on Oct 30, 2016 | root | parent | next [–]

So that "routine designed for that purpose" is how you label a goto destination.

Look, I hate to take this tack, but I know what I'm talking about and in this particular case I'm right and I know it, so I'm going to stop arguing now. I don't mean you any disrespect. It has stopped raining and I have laundry to do so I gotta go.

xapata on Oct 30, 2016 | root | parent | next [–]

Fair enough. BTW, between this and your response about Nutella, you've given one of the most pleasant argument conclusions I've witnessed on this forum.

Edit: I can't resist adding that some languages enforce that every line is numbered/labeled and therefore GOTO can target anything. So there's a bit of quibble room when saying coroutines are equivalent.

carapace on Oct 31, 2016 | root | parent | next [–]

Cheers! That's very gratifying. I try not to be too egomaniacal. :-)

I know what you mean (I used BASIC on a Commodore-64 back in the day.) In the general sense I'm complaining about (hoping to draw attention to) the "foot-gun" aspects of all this, rather that quibble about the details. I wish I could remember and write up the original insight I had that convinced me that coroutines are the same "evil" as goto statements, or find somebody who has written it up. I can remember being convinced, but not the line of argument that convinced me. (Which makes me a little nervous about being so adamant about it here, but what the heck, YOLO.)

gdamjan1 on Oct 30, 2016 | prev | next [–]

It's too late to complain now.

I did like gevent/greenlet a lot, but the wider community, for many years, was unforthcoming to it.

Now asyncio is in the stdlib, including the language changes for coroutines, better than the status-quo.

morty16 on Oct 31, 2016 | prev | next [–]

From early on in the article

> asyncio.get_event_loop() returns the thread bound event loop, it does not return the currently running event loop.

How can these be different objects? In order to ask for the thread-bound event loop, you must be in the thread, right? When/why would you expect anything else?

fyi, I don't have any background with asyncio/twisted.

brettcannon on Oct 31, 2016 | prev | next [–]

If people would like some more illustrative examples for using asyncio there's https://asyncio.readthedocs.io

ddorian43 on Oct 30, 2016 | prev | next [–]

So I guess it's ok for now at least to stay with gevent ? Does asyncio has any pros (except that it's explicit) compared to gevent ?

> rspeer on Oct 30, 2016 | parent | next [–]
>
> People don't write articles named "I don't understand gevent" because nobody expects to understand gevent.
>
> Armin's complaints are for library writers. You will do much better using an async framework with support from the core language (asyncio) than a monkeypatch.
>
> the_mitsuhiko on Oct 30, 2016 | parent | prev | next [–]
>
> asyncio is clearly where the community is moving. Sooner or later everybody in the 3.x world will use it.

woah on Oct 30, 2016 | prev | next [–]

I'm not really that well versed in Python, but wasn't it supposed to be a language for beginners?

> pdonis on Oct 30, 2016 | parent | next [–]
>
> *> wasn't it supposed to be a language for beginners?*
>
> No. It was supposed to be a language in which programmers at widely different skill levels, from beginner to expert, could be productive. Easy to pick up the basics, but also easy to use more advanced techniques when you find you need them.
>
> X86BSD on Oct 30, 2016 | parent | prev | next [–]
>
> I wouldn't go so far as to say it's a language FOR beginners. But it is easier to understand *some* concepts in python than staring at god awful C or Perl or most other languages as a *totally new person to programming*.

_pgmf on Oct 30, 2016 | prev | next [–]

I've been a gevent user for a long time and Python's decision to "bless" twisted by adopting it's patterns was a watershed moment for me, and basically was the beginning of the end of my belief that I'd ever adopt Python 3.

User jerf's comment that asyncio "more than [doubles] the complexity" is absolutely correct. Watch this video of Guido talking about tulip...or struggling to talk about tulip, rather. It's clear the dude is out of his depth and my god the recent changes to the language show that the inmates are now running the asylum... Seems like Python, in it's effort to chase the latest fads, is no longer the language I would endorse to someone new to programming. Whether you think that's a meaningful litmus test or not, the staggering amount of *crap* that's infiltrated the language now completely flies in the face of the zen of python's statement that there should be one and preferably only one way of doing things. Fuck. I'm going to go code some lua now.

https://www.youtube.com/watch?v=1coLC-MUCJc

> hyperbovine on Oct 30, 2016 | parent | next [–]
>
> Oh come on. Hyperbole much? You're acting as if it's impossible to write any sort of meaningful code in 3.x without embracing asyncio. A few 2to3-ish language changes excepted, you can punch the exact same code into the Python 3.x interpreter that you have been for the past 15 years.
>
> > carapace on Oct 30, 2016 | root | parent | next [–]
> >
> > Not much.
> >
> > I agree with coleifer (and others) that Python 3 is losing "the zen". The new f-string stuff is a perfect example: a bad, retrogressive idea that never should have been approved.
> >
> > I have no plans to adopt or use Python 3.
> >
> > > mixmastamyk on Oct 30, 2016 | root | parent | next [–]
> > >
> > > > The new f-string stuff is a perfect example: a bad, retrogressive idea that never should have been approved.
> > >
> > > A convenience that is present in every other modern (and not-so modern) language except python? Hardly.
> > >
> > > tedmiston on Oct 31, 2016 | root | parent | prev | next [–]

I have mixed feelings about f strings. Having the string literal bound to the variable name feels kind of backwards -- that one could break interpolated strings inadvertently by refactoring a variable name. On the other the str.format syntax is a bit verbose, even in shorthand notation.

That said, I use Python 3 every day for almost everything without need to use every shiny new feature.

Too on Oct 31, 2016 | root | parent | next [–]

It's Python we are talking about right? Refactoring a variable name in a non type safe language has always been prone to break something. If you trust your IDE to do refactoring on that level I would also trust it would rename inside the string.

rat87 on Oct 31, 2016 | root | parent | prev | next [–]

Are you kidding?

Interpolated strings are awesome. Only problem is that they weren't introduced in python 3.1 so you could drop the f

stock_toaster on Oct 30, 2016 | parent | prev | next [–]

```
> I've been a gevent user for a long time and Python's
> decision to "bless" twisted by adopting it's patterns
> was a watershed moment for me, and basically was the
> beginning of the end of my belief that I'd ever adopt
> Python 3.
```

This was exactly my experience as well. I had used twisted for quite a while, and never really liked it (aside: my god was inlineCallbacks a great improvement though!). Then later I started using gevent, and used it very successfully for a few years. So much more of a pleasant experience to use.

I still do quite a bit of work with python2. These days I am using Go in quite a few of my personal projects, and for work projects where I am given the choice.

mangeletti on Oct 30, 2016 | parent | prev | next [–]

Hey Charles,

I had the pleasure of working over IRC with you (at Ellington) when I worked for CMG Digital, and I found you to be one of the most knowledgeable Python developers I had met.

But...

I think you're blowing this out of proportion. I think that yes, the implementation details are far too complicated, and yes there has been some serious (IMHO) mistakes made (e.g., type hints, format strings, etc.) in Python 3, but overall the language is terrific and getting better.

ubernostrum on Oct 31, 2016 | parent | prev | next [–]

On the other hand, it's hard to argue with the track record of success of Twisted's approach to async programming in Python. It was at the point where enough people were clamoring for *something* to be blessed that it makes sense to pick the one whose approaches and patterns are as stable and battle-tested as Twisted's.

dschiptsov on Oct 30, 2016 | prev | next [–]

This is what happens when coders are rushing to code without real understanding what they are doing and why. What is worse - they borrow "features" without understanding from amateur JavaScript projects or C#.

What a bloated mess. This is clearly the second system syndrome, described in the Mystical Man-month.

In good old times futures were macros on top of delay and force special forms, and explicit message passing a-la Erlang would do the job.

blakeyrat on Oct 30, 2016 | parent | next [–]

C# does all this stuff *far* clearer and cleaner (and probably more performant).

If they had taken more ideas from C# (especially ExecutionContexts), a lot of his complaints would fade away. He actually explicitly calls this out towards the bottom of the essay.

orf on Oct 30, 2016 | parent | prev | next [–]

What a confused comment. You're saying its bad because it's borrowed hastily from other languages, and a better solution is... borrowing features from Erlang?

dschiptsov on Oct 30, 2016 | root | parent | next [–]

Not at all. Erlang's approach to concurrency has been well-researched and validated (Akka).

Async, await and friends are mere standardized kludges - popular syntactic sugar without clear semantics and real world connection (explicit message passing mimics how biological systems do self-regulation).

So called enterprise languages, especially C++ are full of similar stuff (kludges).

orf on Oct 30, 2016 | root | parent | next [–]

> popular syntactic sugar without clear semantics and real world connection

Uhh... the point of them is that its as close to the semantics of synchronous code as possible. That's the 'real world connection' - your single threaded code can become asynchonous with just a few keyword changes. Rather than "sendRequest()" you do "await sendRequest()".

webmaven on Oct 31, 2016 | parent | prev | next [–]

Second System Syndrome would be if all of this had been jammed into the 3.0 release, *eight years ago*.

It's clear that the async story stumbled into a tarpit *after* that, but I would be very surprised if it wasn't straightened out eventually into a clean syntax & implementation.

Although it will undoubtedly take longer than anyone would like to deprecate the crufty bits.

Oh, and it is *mythical*, not mystical.

dschiptsov on Oct 31, 2016 | root | parent | next [–]

Mythical, of course. Thank you.)

agumonkey on Oct 30, 2016 | prev [–]

I consider A.Ronacher as a very very competent guy. I felt really bad not mastering asyncio; now I'm laughing green.